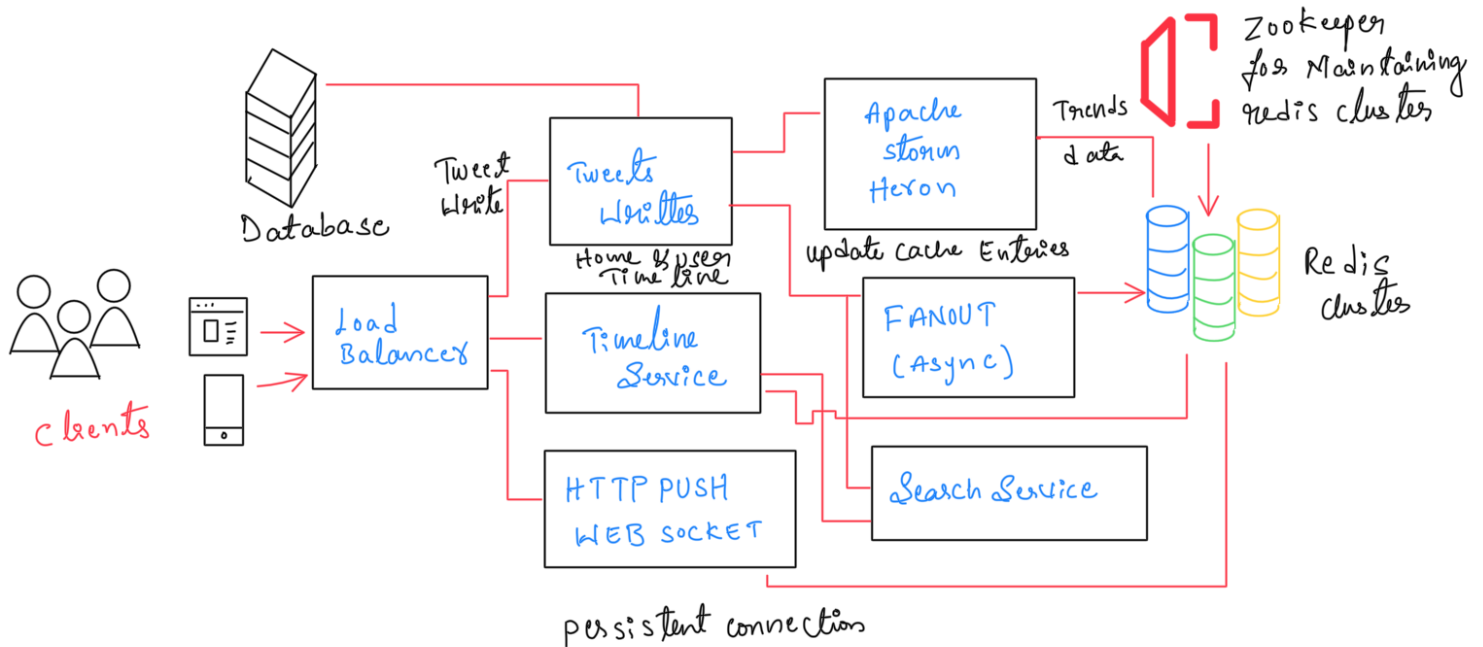


High Level Design for Twitter



Architecture

For twitter We are Exploring the Microservices architecture Since it will make it easier to Horizontally Scale and decouple our Services. Each Service will have Ownership of its own data model. We will divide our System into some Core Services.

User Services

This Service handles user related concern such as authentication and user information. Login page, Signup page, profile page and Home page will be handle into User Services.

Newsfeed Service

This Service will Handle the generation and publishing of user newsfeed. We will discuss about newsfeed in more detail. When it comes to news feed, it seems easy enough to implement But there are a lot of things that can make or break this features. So, lets divide our

Problem into two parts.

→ Generation

Let's assume we want to generate the feed for User A, we will perform the following steps:

- Retrieve the ID's of all the users and the entities (hashtags, topics, etc.)
- Fetch the relevant algorithm to rank the tweets on parameters such as relevance, time management etc.
- Use a Ranking Algorithm to rank the tweets based on parameters such as relevance, time engagement, etc.
- Return the Ranked tweets data to the client in a paginated manner.

Feed generation is an intensive process can take quite a lot of time, especially for users following a lot of people. To improve the performance, the feed can be pre-generated and stored in the cache, then we can have a mechanism to periodically update the feed and apply a ranking algorithm to the new tweets.

Publishing

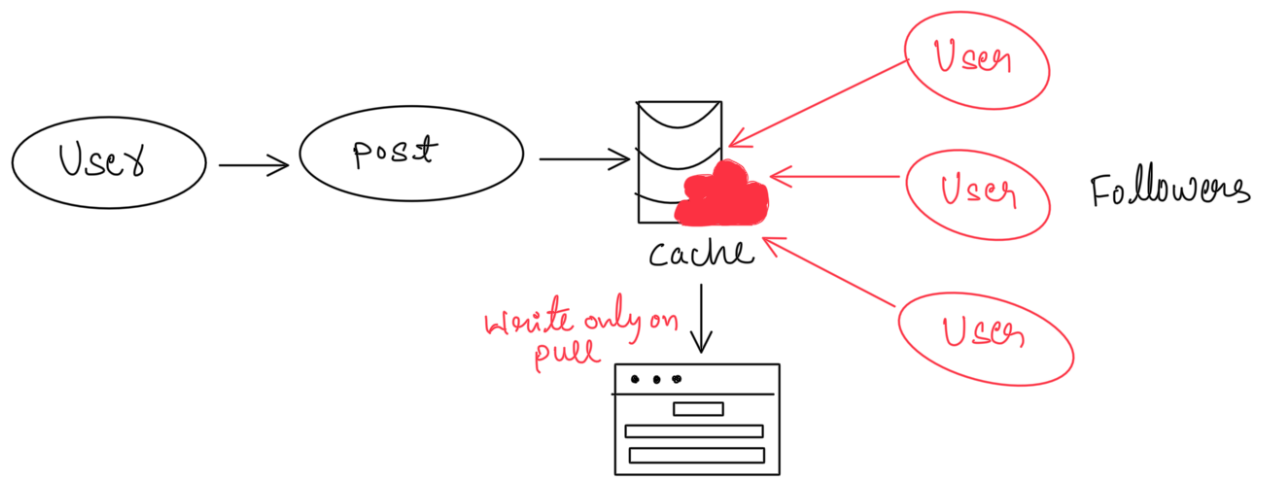
Publishing is the step where the feed data is pushed according to each specific user. This can be a quite heavy operation, as a user may have million of friend or followers, To deal with this, we have three different approaches.

Pull Model (Fan-out on Load)

Feeds are generated only when a user requests them, reducing database writes.

Drawback: Recent tweets appear only on reload, increasing read operations.

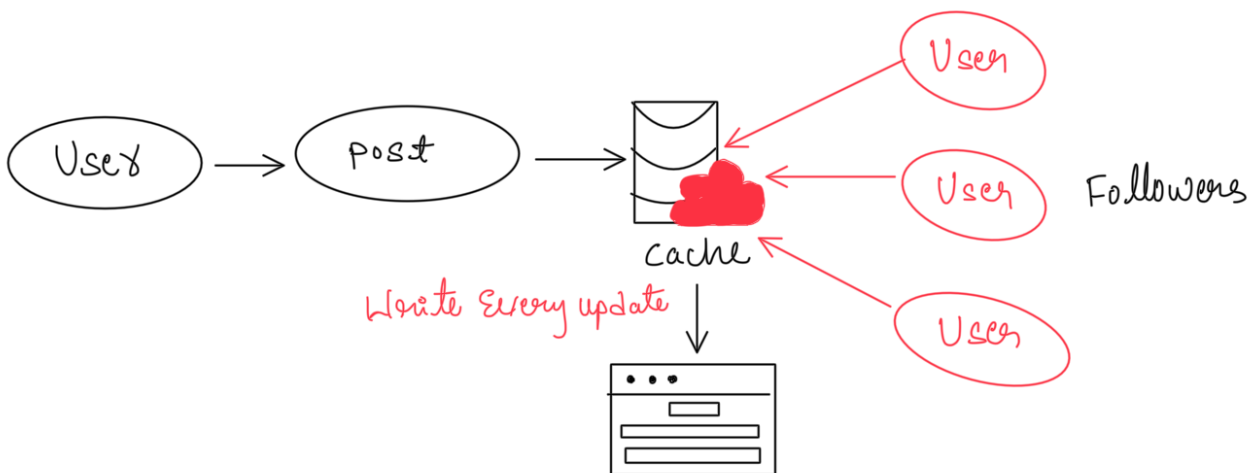
pull model



• push model (Fan-out on Write)

Tweets are immediately pushed to all followers feeds, reducing read time computation.

DrawBack: Increases Write operations on the database



Hybrid Model :

Combines push and pull strategies - Users with few followers use push, while high-followers users (eg celebrities) use pull, Balancing Read and Write Operations

Tweet Service :

The Tweet Service handle tweet-related use case such as posting a tweet, favorites, etc

Retweets

Retweets are one of our extended requirements. To implement this feature, we can simply create a new tweet with the User id of the user retweeting the original tweet and then Modify the type Enum and Content property of the new tweet to link it with the original tweet.

Search Service

This Service is responsible for handling Search related functionality. In Search Service we get the Top post, latest post etc. These things we get because of ranking.

Media Service

This Service will handle the media (images, videos, files etc) uploads.

Analytics Service

This Service will be used for metrics and analytics use cases.

Ranking Algorithm

We will need a ranking algorithm to rank each tweet according to its relevance to each specific user.

Example: Facebook used to utilize an EdgeRank algorithm. Here, the rank of each feed item is described by:

$$\text{Rank} = \text{Affinity} * \text{Weight} * \text{Decay}$$

Where,

- **Affinity** : is the "closeness" of the user to the creator of the edge. If a user frequently likes, comments, or messages the edge creator, then the value of affinity will be higher, resulting in a higher rank for the post.
- **Weight** : is the value assigned according to each edge. A comment can have a higher weightage than likes, and thus a post with more comments is more likely to get a higher rank.
- **Decay** : is the measure of the creation of the edge. The older the edge, the lesser will be the value of decay and eventually the rank.

Now a days, algorithms are much more complex and ranking is done using machine learning